

Descripción de la empresa y problema de negocio

FleetLogix es una **empresa de transporte y logística** que opera una flota de 200 vehículos realizando entregas de última milla en 5 ciudades principales. La empresa ha estado operando con sistemas legacy y hojas de cálculo, pero necesita modernizar su infraestructura de datos para competir en el mercado actual.

Propuesta de solución

Para abordar esta problemática, se propone desarrollar una solución integral de datos dividida en **cuatro etapas consecutivas, denominadas avances**. Cada uno aborda una parte específica de la modernización de la infraestructura de FleetLogix.

Avance 1: Base de datos relacional: establece las bases de la nueva infraestructura de datos a partir del análisis del modelo relacional y la construcción de una base operacional poblada con datos sintéticos.

Tech Stack utilizado

- PostgreSQL 15+
- Python 3.10+
- Jupyter Notebooks
- DBeaver o pgAdmin

Avance 1

Problema de negocio

FleetLogix necesita migrar de sistemas legacy y hojas de cálculo hacia una base de datos robusta que permita gestionar operaciones logísticas a gran escala.

Esta primera etapa plantea construir una base operacional que represente las principales entidades del negocio y sus relaciones. Para ello, se parte de un modelo relacional que debe ser analizado y documentado.

El modelo también debe poblarse con un volumen significativo de datos sintéticos realistas e interrelacionados, garantizando su calidad, integridad y consistencia temporal para los análisis posteriores.

Recursos proporcionados

Para el desarrollo del primer avance se proporcionan los siguientes recursos:

- **fleetlogix_db_schema.sql:** script SQL que contiene la definición del modelo relacional de FleetLogix y sus seis tablas principales.

- **A1-01_data_generation_estudiantes.py:** script base en Python destinado a la generación y carga de datos sintéticos utilizando Faker.
- **FleetLogix_guia_configuracion_inicial.pdf:** documento con las instrucciones y requisitos necesarios para preparar el entorno de trabajo del proyecto.

Desarrollo

El primer avance se desarrolla de manera progresiva, comenzando por la preparación del entorno de trabajo y continuando con el análisis del modelo relacional, la generación masiva de datos sintéticos y, finalmente, la validación de la información obtenida.

El trabajo se divide en cuatro etapas principales:

- **Configuración del entorno:** preparación de PostgreSQL, Python y las dependencias necesarias para ejecutar el proyecto.
- **Análisis del modelo relacional:** estudio de las seis tablas proporcionadas, sus relaciones, constraints, patrones de negocio y posibles mejoras.
- **Generación y carga de datos sintéticos:** adaptación y ejecución del script Python encargado de poblar las tablas respetando las reglas del negocio.
- **Control de calidad:** validación de integridad referencial, consistencia temporal y coherencia de los datos generados.

1. Configuración del entorno

Siguiendo *FleetLogix_guia_configuracion_inicial.pdf*, se utiliza **PostgreSQL** como sistema gestor de la base de datos y **Python** para los procesos de generación y tratamiento de datos. Se crea la base fleetlogix y se configura su acceso para permitir posteriormente la conexión desde Python.

Para administrar y consultar la base se utiliza **DBeaver**, desde donde se ejecutan los scripts SQL, se inspeccionan las tablas y se verifican los resultados obtenidos.

Finalmente, se instalan las dependencias de Python requeridas por el proyecto: psycopg2-binary, pandas, numpy, Faker, tabulate, Jupyter y notebook.

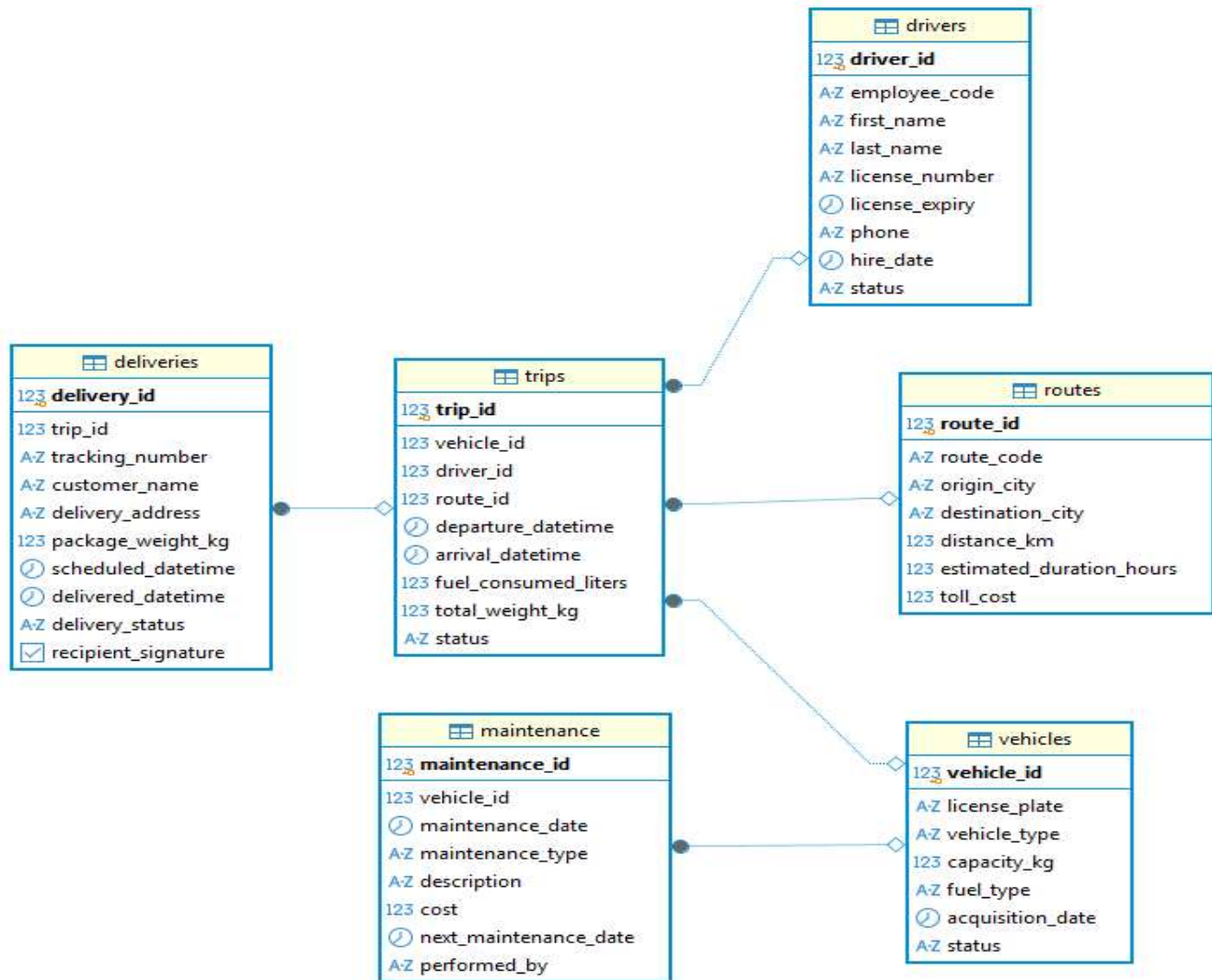
2. Implementación y análisis del modelo relacional

Una vez configurado el entorno, se implementó el modelo relacional proporcionado ejecutando el script fleetlogix_db_schema.sql sobre la base de datos fleetlogix. Como resultado, se crearon las seis tablas que representan las principales entidades de la operación: vehicles, drivers, routes, trips, deliveries y maintenance.

La estructura permite centralizar la información de la flota, los conductores y las rutas, vinculándola con los viajes, las entregas y el historial de mantenimiento de los vehículos.

2.1 Diagrama entidad-relación

A partir del modelo implementado se generó el siguiente diagrama entidad-relación (ER), que permite visualizar las tablas, sus atributos y las relaciones definidas entre ellas.



2.2 Análisis de las tablas

El modelo relacional está compuesto por seis tablas que representan las principales entidades y procesos de la operación logística. A continuación, se describe la función de cada una y su estructura.

- La tabla **vehicles** almacena la información de los vehículos que componen la flota de FleetLogix y sus principales características operativas.

Campo	Tipo de dato	Restricciones	Descripción
vehicle_id	SERIAL	PRIMARY KEY	Identificador único del vehículo.
license_plate	VARCHAR(20)	UNIQUE, NOT NULL	Patente del vehículo.
vehicle_type	VARCHAR(50)	NOT NULL	Tipo de vehículo.
capacity_kg	DECIMAL(10,2)	—	Capacidad de carga en kilogramos.
fuel_type	VARCHAR(20)	—	Tipo de combustible utilizado.
acquisition_date	DATE	—	Fecha de adquisición del vehículo.
status	VARCHAR(20)	DEFAULT 'active'	Estado operativo del vehículo.

- La tabla **drivers** almacena la información de los conductores de FleetLogix, incluyendo sus datos de identificación, licencia y situación laboral.

Campo	Tipo de dato	Restricciones	Descripción
driver_id	SERIAL	PRIMARY KEY	Identificador único del conductor.
employee_code	VARCHAR(20)	UNIQUE, NOT NULL	Código único asignado al empleado.

first_name	VARCHAR(50)	NOT NULL	Nombre del conductor.
last_name	VARCHAR(50)	NOT NULL	Apellido del conductor.
license_number	VARCHAR(50)	UNIQUE, NOT NULL	Número único de licencia de conducir.
license_expiry	DATE	—	Fecha de vencimiento de la licencia.
phone	VARCHAR(20)	—	Número de teléfono del conductor.
hire_date	DATE	—	Fecha de contratación.
status	VARCHAR(20)	DEFAULT 'active'	Estado laboral u operativo del conductor.

La tabla **routes** almacena las rutas utilizadas por FleetLogix, definiendo las ciudades de origen y destino junto con información relevante para la planificación de los viajes.

Campo	Tipo de dato	Restricciones	Descripción
route_id	SERIAL	PRIMARY KEY	Identificador único de la ruta.
route_code	VARCHAR(20)	UNIQUE, NOT NULL	Código único asignado a la ruta.
origin_city	VARCHAR(100)	NOT NULL	Ciudad de origen de la ruta.
destination_city	VARCHAR(100)	NOT NULL	Ciudad de destino de la ruta.
distance_km	DECIMAL(10,2)	—	Distancia de la ruta en kilómetros.
estimated_duration_hours	DECIMAL(5,2)	—	Duración estimada del recorrido en horas.
toll_cost	DECIMAL(10,2)	DEFAULT 0	Costo de peajes asociado a la ruta.

- La tabla **trips** registra los viajes realizados por FleetLogix, vinculando cada operación con el vehículo, conductor y ruta correspondientes, además de almacenar información sobre tiempos, carga y consumo de combustible.

Campo	Tipo de dato	Restricciones	Descripción
trip_id	SERIAL	PRIMARY KEY	Identificador único del viaje.
vehicle_id	INTEGER	FOREIGN KEY	Vehículo asignado al viaje.
driver_id	INTEGER	FOREIGN KEY	Conductor asignado al viaje.
route_id	INTEGER	FOREIGN KEY	Ruta correspondiente al viaje.
departure_datetime	TIMESTAMP	NOT NULL	Fecha y hora de salida.
arrival_datetime	TIMESTAMP	—	Fecha y hora de llegada.
fuel_consumed_liters	DECIMAL(10,2)	—	Combustible consumido durante el viaje.
total_weight_kg	DECIMAL(10,2)	—	Peso total transportado en kg.
status	VARCHAR(20)	DEFAULT 'in_progress'	Estado del viaje.

- La tabla **deliveries** registra las entregas asociadas a los viajes de FleetLogix, incluyendo información del destinatario, paquete, programación y estado de la entrega.

Campo	Tipo de dato	Restricciones	Descripción
delivery_id	SERIAL	PRIMARY KEY	Identificador único de la entrega.
trip_id	INTEGER	FOREIGN KEY	Viaje al que pertenece la entrega.
tracking_number	VARCHAR(50)	UNIQUE, NOT NULL	Número único de seguimiento.
customer_name	VARCHAR(100)	NOT NULL	Nombre del cliente.
delivery_address	TEXT	NOT NULL	Dirección de entrega.
package_weight_kg	DECIMAL(10,2)	—	Peso del paquete en kilogramos.
scheduled_datetime	TIMESTAMP	—	Fecha y hora programada para la entrega.

delivered_datetime	TIMESTAMP	—	Fecha y hora en que se realizó la entrega.
delivery_status	VARCHAR(20)	DEFAULT 'pending'	Estado de la entrega.
recipient_signature	BOOLEAN	DEFAULT FALSE	Indica si se registró la firma del destinatario.

- La tabla ***maintenance*** registra los mantenimientos realizados a los vehículos de FleetLogix, incluyendo el tipo de intervención, fechas, costos y responsable.

Campo	Tipo de dato	Restricciones	Descripción
maintenance_id	SERIAL	PRIMARY KEY	Identificador único del mantenimiento.
vehicle_id	INTEGER	FOREIGN KEY	Vehículo al que corresponde el mantenimiento.
maintenance_date	DATE	NOT NULL	Fecha en que se realizó el mantenimiento.
maintenance_type	VARCHAR(100)	NOT NULL	Tipo de mantenimiento realizado.
description	TEXT	—	Descripción del trabajo realizado.
cost	DECIMAL(10,2)	—	Costo del mantenimiento.
next_maintenance_date	DATE	—	Fecha prevista para el próximo mantenimiento.
performed_by	VARCHAR(100)	—	Responsable de realizar el mantenimiento.

2.3 Relaciones del modelo

El modelo establece cinco relaciones mediante claves foráneas, permitiendo conectar los datos operativos y mantener la integridad referencial entre las tablas.

Relación	Clave foránea	Cardinalidad	Descripción
vehicles → trips	trips.vehicle_id	1:N	Un vehículo puede estar asociado a múltiples viajes, mientras que cada viaje referencia un vehículo.
drivers → trips	trips.driver_id	1:N	Un conductor puede realizar múltiples viajes, mientras que cada viaje referencia un conductor.
routes → trips	trips.route_id	1:N	Una ruta puede utilizarse en múltiples viajes, mientras que cada viaje referencia una ruta.
trips → deliveries	deliveries.trip_id	1:N	Un viaje puede incluir múltiples entregas, mientras que cada entrega pertenece a un viaje.
vehicles → maintenance	maintenance.vehicle_id	1:N	Un vehículo puede registrar múltiples mantenimientos, mientras que cada mantenimiento corresponde a un vehículo.

Estas relaciones permiten vincular los viajes con los recursos utilizados y las operaciones que dependen de ellos. **“trips”** ocupa una posición central en el modelo al relacionar vehículos, conductores y rutas, y servir a su vez como referencia para las entregas.

2.4 Constraints del modelo

El modelo utiliza diferentes constraints para garantizar la identificación de los registros, evitar duplicados y mantener la integridad de las relaciones entre las tablas.

Constraint	Aplicación en el modelo	Función
PRIMARY KEY	vehicle_id, driver_id, route_id, trip_id, delivery_id, maintenance_id	Identifica de manera única cada registro de las seis tablas.
FOREIGN KEY	trips.vehicle_id, trips.driver_id, trips.route_id, deliveries.trip_id, maintenance.vehicle_id	Mantiene la integridad referencial, evitando referencias a registros inexistentes.
UNIQUE	vehicles.license_plate, drivers.employee_code, drivers.license_number, routes.route_code, deliveries.tracking_number	Evita duplicados en atributos que deben ser únicos dentro del sistema.
NOT NULL	Aplicado a campos obligatorios de identificación y operación	Impide almacenar registros sin determinados datos considerados necesarios por el modelo.
DEFAULT	Estados de vehículos, conductores, viajes y entregas; costo de peajes y firma del destinatario	Asigna automáticamente un valor inicial cuando el campo no se especifica al insertar un registro.

En conjunto, estos constraints permiten mantener la consistencia de la información almacenada, garantizando la identificación única de los registros, la integridad de las relaciones entre las tablas y el cumplimiento de los campos definidos como obligatorios.

2.5 Patrones de negocio implícitos

La estructura del modelo permite identificar distintos patrones propios de la operación logística de FleetLogix.

Cada viaje vincula un vehículo, un conductor y una ruta, reflejando que una operación de transporte requiere la asignación conjunta de estos tres recursos. Esta relación permite conocer qué unidad fue utilizada, quién realizó el viaje y qué recorrido se efectuó.

Las entregas dependen de los viajes, permitiendo que una misma operación agrupe múltiples entregas y manteniendo la trazabilidad entre los pedidos y el recorrido realizado.

El modelo contempla variables operativas como la capacidad de los vehículos, el peso transportado, la distancia de las rutas, la duración estimada y el consumo de combustible. Estos atributos permiten representar restricciones y características propias de la operación logística.

Los registros de mantenimiento se asocian directamente a los vehículos, permitiendo conservar el historial de intervenciones de cada unidad.

Finalmente, las fechas y estados permiten representar la evolución temporal de la operación y diferenciar situaciones como vehículos activos, viajes en progreso o entregas pendientes y completadas.

2.6 Posibles mejoras del modelo

El análisis del modelo permite identificar mejoras orientadas a reforzar la calidad y consistencia de los datos, sin modificar la estructura base utilizada en el proyecto.

- **Validación de valores numéricos:** Incorporar restricciones CHECK para evitar valores que no tienen sentido operativo. Por ejemplo, la capacidad de un vehículo, el peso de una entrega, la distancia de una ruta o el costo de un mantenimiento no deberían admitir valores negativos.

- **Control de estados:** Los campos de estado están definidos como texto y aceptan cualquier valor. Limitar las opciones disponibles evita registros inconsistentes y garantiza que vehículos, conductores, viajes y entregas utilicen estados previamente definidos.
- **Coherencia temporal:** Las fechas deben respetar el orden lógico de cada proceso. La llegada de un viaje debe ser posterior a su salida y la fecha del próximo mantenimiento debe ser posterior al mantenimiento realizado.
- **Campos obligatorios:** Algunos datos relevantes actualmente admiten valores nulos. La capacidad del vehículo, la distancia y duración estimada de las rutas y el vencimiento de la licencia del conductor deberían registrarse obligatoriamente para poder aplicar correctamente las reglas operativas.
- **Condiciones operativas:** La existencia de un vehículo o conductor no significa necesariamente que esté habilitado para realizar un viaje. La asignación debe considerar el estado del vehículo y la vigencia de la licencia del conductor.
- **Compatibilidad entre conductor y vehículo:** Incorporar el tipo o categoría de licencia del conductor (license_type) permite verificar que cada conductor esté legalmente habilitado para manejar el vehículo que se le asigna.

3. Generación y carga de datos sintéticos

La generación de datos se realiza mediante el script **A1-01_data_generation_estudiantes.py**, diseñado para simular una operación logística coherente en Colombia. Los valores no se generan de forma independiente: cada tabla utiliza información de las entidades relacionadas para construir registros consistentes.

El proceso debe generar 200 vehículos, 400 conductores, 50 rutas, 100.000 viajes, 400.000 entregas y 5.000 registros de mantenimiento, alcanzando un volumen total superior a 505.000 registros.

3.1 Resultados de la primera ejecución

Una vez completadas las secciones pendientes del script, se realizó una primera ejecución completa y se verificó la cantidad de registros almacenados en cada tabla. Los resultados obtenidos fueron los siguientes:

Tabla	Registros
vehicles	200
drivers	400
routes	48
trips	100.000
deliveries	400.001
maintenance	4.920

Si bien el volumen total supera los 505.000 registros, routes, deliveries y maintenance no alcanzaron la cantidad esperada, por lo que fue necesario revisar la lógica de generación de estas tablas para su ajuste.

3.2 Ajustes realizados al script

A partir de los resultados obtenidos en la primera ejecución, se revisó la lógica de generación para corregir las diferencias detectadas y asegurar la consistencia de los datos.

Generación de rutas

El script generaba 48 rutas debido a la cantidad de recorridos asignados a cada combinación de ciudades. Se ajustó la distribución incorporando una tercera alternativa para los recorridos entre Medellín y Cali, en ambos sentidos. De esta forma, se alcanzaron las 50 rutas esperadas.

Generación de entregas

La condición de corte permitía generar una entrega adicional antes de finalizar el proceso, obteniendo 400.001 registros. Se modificó la condición para detener la generación al alcanzar el límite, obteniendo exactamente 400.000 entregas.

Período de los viajes

El intervalo utilizado para distribuir los viajes convertía los minutos a un número entero, reduciendo el período efectivo de generación. Se eliminó esta conversión para conservar los decimales y distribuir los 100.000 viajes a lo largo de los dos años completos de operación.

Distancias entre ciudades

La función de distancias ordenaba alfabéticamente las ciudades antes de realizar la búsqueda, pero algunas claves del diccionario no seguían ese mismo criterio. Se corrigieron las claves para obtener la distancia correspondiente independientemente del sentido del recorrido.

Registros de mantenimiento

La cantidad de mantenimientos se calcula individualmente para cada vehículo, generando uno por cada 20 viajes completos. Por este motivo se obtuvieron 4.913 registros. Los viajes restantes de cada vehículo no justifican un nuevo mantenimiento, por lo que se mantuvo esta lógica sin forzar el total a 5.000.

3.3 Resultados de la ejecución final

Una vez aplicadas las correcciones, se ejecutó nuevamente el proceso completo de generación. Los resultados obtenidos fueron:

Tabla	Registros generados
vehicles	200
drivers	400
routes	50
trips	100.000
deliveries	400.000
maintenance	4.913
Total	505.563

3.4 Lógica de generación y justificación de tablas

La generación de datos sigue la estructura relacional del modelo, utilizando información de las tablas previamente creadas para mantener coherencia entre los registros.

Vehículos

La tabla vehicles genera la flota utilizando cuatro tipos de vehículos: Camión Grande, Camión Mediano, Van y Motocicleta. Cada uno posee una capacidad de carga, tipo de combustible, fecha de adquisición y estado. La capacidad se utiliza posteriormente para limitar el peso transportado en los viajes.

Conductores

La tabla drivers genera información ficticia de los conductores, incluyendo código de empleado, datos personales, licencia, fecha de vencimiento, contratación y estado. Para generar los viajes se seleccionan conductores con estado activo.

Rutas

La tabla routes genera recorridos entre Bogotá, Medellín, Cali, Barranquilla y Cartagena. Cada ruta contiene origen, destino, distancia, duración estimada y costo de peajes. La distancia y duración son utilizadas posteriormente para calcular características de los viajes.

Viajes

La tabla trips relaciona las tres entidades anteriores. Mediante generate_trips() se seleccionan vehículos activos, conductores activos y rutas existentes para generar 100.000 viajes distribuidos a lo largo de dos años.

La duración del viaje toma como referencia la duración estimada de la ruta; el consumo de combustible se calcula según la distancia recorrida; y el peso transportado se genera entre el 40 % y el 90 % de la capacidad del vehículo. La hora de salida se determina mediante _get_hourly_distribution(), permitiendo representar distintos niveles de actividad durante el día.

Entregas

La tabla deliveries se genera a partir de los viajes existentes. Cada viaje contiene entre 2 y 6 entregas, siendo 4 la cantidad más probable. El peso de los paquetes se distribuye utilizando aproximadamente el 95 % del peso transportado y sus horarios se calculan en relación con los tiempos del viaje.

Mantenimiento

La tabla maintenance utiliza el historial de viajes de cada vehículo para generar aproximadamente un mantenimiento por cada 20 viajes completos. Se contemplan diferentes intervenciones, como cambio de aceite, revisión de frenos, cambio de llantas, mantenimiento general, revisión de motor y alineación y balanceo.

3.5 Método generate_trips() y distribución horaria

El método generate_trips() es responsable de construir el historial de viajes. Para hacerlo, consulta las tablas vehicles, drivers y routes, obteniendo los vehículos activos junto con su capacidad, los conductores activos y las rutas disponibles. A partir de estos datos genera cada registro de trips, manteniendo las relaciones entre las tablas.

Los viajes se distribuyen progresivamente a lo largo de aproximadamente 730 días. Para cada uno se selecciona un vehículo, un conductor y una ruta. La información seleccionada se utiliza para calcular variables relacionadas: la duración se basa en el tiempo estimado de la ruta, el consumo de combustible en la distancia recorrida y el peso transportado en la capacidad del vehículo. La fecha de llegada se obtiene a partir de la salida y de la duración calculada.

El método auxiliar _get_hourly_distribution() se utiliza para evitar que las horas de salida tengan la misma probabilidad durante todo el día. Se establece una actividad menor durante las horas de baja demanda, una mayor actividad entre las 6:00 y las 20:00 y dos franjas de mayor concentración: entre las 8:00 y las 12:00 y entre las 14:00 y las 18:00.

De esta manera, `generate_trips()` combina las relaciones del modelo con una distribución temporal más realista, generando viajes coherentes tanto con los recursos disponibles como con los horarios habituales de una operación logística.

4. Control de Calidad

Una vez finalizada la generación y carga de los datos, se utiliza el script `validacion_calidad_datos.sql` para verificar la calidad y consistencia de la información almacenada en la base de datos.

El script realiza tres grupos de controles:

4.1 Cantidad de registros

Se verifica individualmente la cantidad de registros de cada tabla para comprobar los resultados obtenidos durante la generación.

4.2 Integridad referencial

Se comprueba que los viajes estén asociados a vehículos, conductores y rutas existentes; que las entregas pertenezcan a viajes registrados; y que los mantenimientos correspondan a vehículos existentes.

4.3 Consistencia temporal

Se verifica que la fecha y hora de llegada de cada viaje sea posterior a su fecha y hora de salida, permitiendo detectar posibles inconsistencias temporales.

Estos controles permiten validar los datos generados antes de utilizarlos en las etapas posteriores de análisis.

4.4 Resultados de la validación

La ejecución de `validacion_calidad_datos.sql` no detectó inconsistencias. Todos los controles de integridad referencial y consistencia temporal devolvieron 0 registros, indicando que no existen relaciones inválidas entre las tablas ni viajes cuya fecha de llegada sea anterior o igual a la fecha de salida.

Los conteos de las tablas también confirmaron los resultados obtenidos durante la generación: 200 vehículos, 400 conductores, 50 rutas, 100.000 viajes, 400.000 entregas y 4.913 registros de mantenimiento, alcanzando un total de 505.563 registros.

5. Mejoras futuras

El modelo y los datos generados permiten representar la operación principal de FleetLogix y constituyen una base para futuros análisis. A partir del trabajo realizado se identifican oportunidades de mejora tanto técnicas como orientadas al negocio.

5.1 Mejoras del modelo y generación de datos

- Incorporar el tipo de licencia del conductor y definir su compatibilidad con los distintos tipos de vehículos.
- Incorporar controles de disponibilidad de conductores para evitar la asignación del mismo conductor a viajes con horarios superpuestos.
- Refinar el cálculo del consumo de combustible considerando el tipo de vehículo, la distancia recorrida y la carga transportada.

- Mejorar la lógica de entregas demoradas para representar con mayor precisión el cumplimiento de los horarios programados.
- Ampliar las validaciones automáticas del script Python para cubrir todas las relaciones del modelo.
- Revisar los parámetros de generación de rutas y mantenimientos para permitir un mayor control sobre los volúmenes generados.

5.2 Mejoras orientadas al negocio

- Incorporar información sobre costos operativos de cada viaje para analizar la rentabilidad de rutas y operaciones.
- Registrar tarifas o ingresos asociados a las entregas para comparar ingresos y costos.
- Incorporar información geográfica más detallada para analizar recorridos, zonas de entrega y eficiencia de las rutas.
- Registrar causas de retrasos y entregas fallidas para identificar problemas recurrentes.
- Incorporar métricas de utilización de los vehículos para detectar unidades con baja utilización o sobrecarga operativa.
- Analizar el historial de mantenimiento junto con kilometraje, viajes realizados y costos para avanzar hacia estrategias de mantenimiento preventivo.
- Incorporar indicadores de desempeño de conductores, rutas y vehículos que permitan comparar eficiencia, cumplimiento y costos.